Prática de Laboratório 01: Introdução à Análise de Algoritmos

Disciplina: Análise de Algoritmos

Instruções Gerais

- Formem duplas.
- Vocês precisarão escrever e executar códigos em Python. Podem utilizar a IDE instalada nas máquinas do laboratório ou o compilador online: `https://www.online-python.com/`.
- O objetivo não é apenas rodar o código, mas **analisar** os tempos de execução impressos no terminal e responder às perguntas propostas.

---

1 Missão 1: A Corrida Assintótica ($O(n)$ vs $O(n^2)$)

Nesta etapa, vamos observar empiricamente como o tempo de execução escala conforme o tamanho da instância cresce, validando os limites teóricos discutidos em sala.

Passo 1: Copiem e executem o código abaixo. Ele possui dois algoritmos: um de complexidade linear e outro quadrático.

```
import time

def algoritmo_linear(n):
    soma = 0
    for i in range(n):
        soma += 1

def algoritmo_quadratico(n):
    soma = 0
    for i in range(n):
        for j in range(n):
            soma += 1

# Bateria de testes
tamanhos = [1000, 5000, 10000]

print("--- Testando Algoritmo Linear O(n) ---")
for n in tamanhos:
    inicio = time.time()
    algoritmo_linear(n)
    fim = time.time()
    print(f"n = {n:5d} | Tempo: {fim - inicio:.5f} segundos")

print("\n--- Testando Algoritmo Quadr tico O(n^2) ---")
for n in tamanhos:
```

```
    inicio = time.time()
    algoritmo_quadratico(n)
    fim = time.time()
    print(f"n = {n:5d} | Tempo: {fim - inicio:.5f} segundos")
```

Análise (Discutam e anotem)

1. Observem os tempos impressos. Quando o valor de n passou de 1.000 para 10.000 (multiplicou por 10), o que aconteceu com o tempo do algoritmo *linear*? E com o tempo do algoritmo *quadrático*?

2. Adicionem o valor $\mathbf{n = 100000}$ na lista `tamanhos` e executem novamente. O que ocorreu? Baseado no crescimento observado, por que algoritmos $O(n^2)$ são considerados inviáveis para grandes volumes de dados?

---

2 Missão 2: Caçando o Pior e o Melhor Caso

Sabemos que o tempo de execução pode variar drasticamente entre instâncias do mesmo tamanho. Vamos testar isso na prática com um algoritmo de busca linear.

Passo 1: Executem o código abaixo. Ele cria uma instância de vetor contendo dez milhões de elementos e faz duas buscas distintas.

```
import time

# Criando uma instancia: vetor de 0 a 9.999.999
print("Gerando vetor na memoria...")
vetor_gigante = list(range(10000000))
print("Vetor gerado!\n")

def busca(lista, alvo):
    for elemento in lista:
        if elemento == alvo:
            return True
    return False

# Cen rio A: Buscando o n mero 0
inicio = time.time()
busca(vetor_gigante, 0)
fim = time.time()
print(f"Tempo Cen rio A (Buscando 0): {fim - inicio:.6f} s")

# Cen rio B: Buscando o n mero -1
inicio = time.time()
busca(vetor_gigante, -1)
fim = time.time()
print(f"Tempo Cen rio B (Buscando -1): {fim - inicio:.6f} s")
```

Análise (Discutam e anotem)

1. Ambos os testes operam sobre a exata mesma instância (o mesmo `vetor_gigante` de tamanho $n = 10^7$). Houve diferença no tempo de execução? Por quê?

2. Qual cenário representa o $T_{min}(n)$ (Melhor Caso) e qual representa o $T_{max}(n)$ (Pior Caso)? Justifique com base no comportamento do laço `for` na função de busca.

---

3 Missão 3: Identificando o Gargalo ($O(n) + O(n^2) = ?$)

Em sistemas reais, os algoritmos são compostos por várias etapas. A regra da Análise Assintótica diz que o termo de maior grau sempre domina o tempo total para um n suficientemente grande. Vamos visualizar esse "domínio".

Passo 1: O código a seguir possui duas etapas. A Etapa 1 executa 500 vezes a quantidade de instâncias ($O(n)$ com constante alta). A Etapa 2 é puramente quadrática ($O(n^2)$).

```python
import time

def etapa1_linear(n):
    soma = 0
    # O laco roda 500 vezes o valor de n
    for i in range(n * 500):
        soma += 1

def etapa2_quadratica(n):
    soma = 0
    for i in range(n):
        for j in range(n):
            soma += 1

def programa_completo(n):
    print(f"--- Testando para n = {n} ---")

    t0 = time.time()
    etapa1_linear(n)
    t1 = time.time()

    etapa2_quadratica(n)
    t2 = time.time()

    print(f"Tempo Etapa 1 (Linear): {t1 - t0:.5f} s")
    print(f"Tempo Etapa 2 (Quadratica): {t2 - t1:.5f} s")
    print(f"Tempo Total: {t2 - t0:.5f} s\n")

# Vamos testar valores progressivos
programa_completo(100)
programa_completo(1000)
programa_completo(5000)
```

Análise (Discutam e anotem)

1. Para $n = 100$, qual das duas etapas demorou mais?
2. Para $n = 5000$, qual etapa dominou completamente o tempo total de execução?
3. Se este fosse um sistema real rodando em produção e estivesse travando devido à lentidão, de nada adiantaria otimizar o algoritmo da "Etapa 1". Baseado no que você observou sobre limites assintóticos, por que otimizar gargalos quadráticos deve ser sempre a prioridade em análise de algoritmos?